I. Latent Diffusion Model (LDM)

Well curated network datasets are largely unavailable in the public domain, motivating researchers to adopt generative AI based approaches for synthetic sample generation. GAN based methods have been used in [1] and [2] to generate IP and QUIC network packets from noise. However, GANs are difficult to train due to parameter oscillation, which destabilizes the network and delays convergence [3]. In [4], synthetic packets are generated using GPT 3, but its high computational and memory requirements make it unsuitable for resource constrained MQTT based IoT environments [5]. Deep reinforcement learning has also been applied in [6] for adversarial packet generation using raw malicious packets as input. The works in [7], [8], and [5] utilize diffusion models in the pixel domain for packet generation across different protocol layers. Since high dimensional pixel spaces are computationally expensive for constrained IoT systems, we adopt a latent diffusion model (LDM).

The diffusion model operates by gradually adding noise to the original data and subsequently removing it to recover the original sample (refer to Fig. 1). The process of progressively adding small amounts of Gaussian noise to a clean sample X_0 is known as forward diffusion, while backward diffusion performs denoising by learning the reverse transitions and successively removing noise to reconstruct X_0 from the noisy sample X_T.

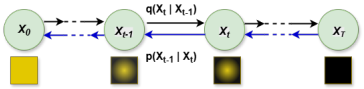

Fig. 1: Diffusion Process

As shown in Fig. 1, a sequence of noisy latent variables from X_0 to X_T is generated through a Markov process, where a small amount of Gaussian noise is added at each time step using Eqn. 1. Here, X_t represents the noisy latent at time step t, while X_{t-1} denotes the latent variable from the previous step. The parameter β_t controls the amount of noise added at each step. According to the Gaussian distribution definition [9], $\sqrt{1-\beta_t}X_{t-1}$ represents the mean and $\beta_t I$ denotes the variance, where I is the identity matrix.

$$q(X_t|X_{t-1}) = \mathcal{N}(X_t; \sqrt{1-\beta_t}X_{t-1}, \beta_t I) \tag{1}$$

The backward diffusion process is given by Eqn. 2. Here, $\mu_\theta(X_t, t)$ is the predicted mean by the backward diffusion model neural network,$\Sigma_\theta(X_t, t)$ is the predicted variance, and θ is the parameter of the neural network trained for denoising.

$$p(X_{t-1}|X_t) = \mathcal{N}(X_{t-1}; \mu_\theta(X_t, t), \Sigma_\theta(X_t, t)) \tag{2}$$

Generally, these models need high-dimensional data spaces; however, the LDM [10] also works with a low-dimensional latent space. The latent space is formed using autoencoders, which compress the high-dimensional input data into a low-dimensional compressed space. The overall idea is to add noise to this latent space through forward diffusion, and then predict the noise added at each step using backward diffusion. The backward diffusion process minimizes the loss calculated using the mean squared error loss in Eqn. 3, which measures the difference between the actual noise ξ and the predicted noise $\xi_0(X_t, t)$ across different timesteps t, where X_t is the noisy latent at diffusion step t.

$$L = E_{X_0,\xi,t}[||\xi - \xi_0(X_t, t)||_2^2] \tag{3}$$

In the original design [10] of the LDM, the authors also use the conditioning mechanism, which helps with the conditional synthesis of images based on a variety of texts. In our case, we are generating CSV data from an existing CSV file, and do not require the conditioning mechanism. We use LDM to generate synthetic datasets for *MQTTSec* (details in Sec. **??**).

II. Reinforcement Learning

Reinforcement Learning (RL) [11] is a branch of AI in which an agent learns decision making through interaction with an environment to maximize cumulative rewards. Unlike supervised learning, RL follows a trial and error approach, where the agent observes a state, performs an action, receives a reward and a new state, and continuously improves its policy based on feedback. RL is commonly modeled as a Markov Decision Process (MDP).

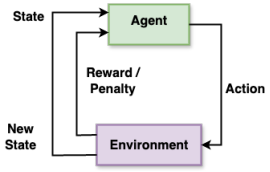

Fig. 2: Reinforcement Learning

Fig. 2 illustrates the basic architecture of RL, where the agent interacts with the environment by taking actions, receiving rewards or penalties along with a new state, which subsequently influences future actions.

The goal of RL is to learn an optimal policy that maximizes the expected return. State action quality can be estimated using Q learning, policy gradient, and Monte Carlo methods.

Q-learning enables an agent to learn optimal decisions through interaction with the environment. The experiences are stored in a tabular structure called a Q-table, where the Q-values are updated as [11]:

$$Q(s, a) = R(s, a) + \gamma max_a Q(s', a) \tag{4}$$

Here, $Q(s, a)$ is the Q-value for given state-action pair, $R(s, a)$ is the reward for taking action a in state s, γ is the discount factor, and $max_a Q'(s', a)$ is the maximum Q-value for the next state s' and action a.

Policy gradient methods directly optimize policy parameters by estimating the expected return gradient [11]. These methods are effective for continuous or high dimensional action spaces where value based methods like Q-learning may struggle.

The Monte Carlo method estimates state or action values using the actual returns obtained after complete episodes, without bootstrapping as in Q-learning. It is well suited for episodic tasks, and in *MQTTSec*, we employ the Monte Carlo mechanism.

REFERENCES

[1] A. Cheng, "Pac-gan: Packet generation of network traffic using generative adversarial networks," in *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*. IEEE, 2019, pp. 0728–0734.

[2] D. Dey and N. Ghosh, "iquic: An intelligent framework for defending quic connection id-based dos attack using advantage actor-critic rl," *Computers & Security*, p. 104463, 2025.

[3] H. Thanh-Tung and T. Tran, "Catastrophic forgetting and mode collapse in gans," in *2020 international joint conference on neural networks (ijcnn)*. IEEE, 2020, pp. 1–10.

[4] D. K. Kholgh and P. Kostakos, "Pac-gpt: A novel approach to generating synthetic network traffic with gpt-3," *IEEE Access*, vol. 11, pp. 114 936–114 951, 2023.

[5] T. An, Y. Zhou, H. Zou, and J. Yang, "Iot-llm: Enhancing real-world iot task reasoning with large language models," *arXiv preprint arXiv:2410.02429*, 2024.

[6] S. Hore, J. Ghadermazi, D. Paudel, A. Shah, T. Das, and N. Bastian, "Deep packgen: A deep reinforcement learning framework for adversarial network packet generation," *ACM Transactions on Privacy and Security*, vol. 28, no. 2, pp. 1–33, 2025.

[7] S. Zhang, H. Chai, Y. Li, B. Qiu, L. Yue, and R. Pan, "Packetdiff: A flow guided diffusion model for network packet trace generation," *IEEE Internet of Things Journal*, 2025.

[8] A. Bin Jasni, A. Manada, and K. Watabe, "Diffupac: Contextual mimicry in adversarial packets generation via diffusion model," *Advances in Neural Information Processing Systems*, vol. 37, pp. 133 846–133 878, 2024.

[9] T. B. Farver, "Chapter 1 - concepts of normality in clinical biochemistry," in *Clinical Biochemistry of Domestic Animals (Sixth Edition)*, sixth edition ed., J. J. Kaneko, J. W. Harvey, and M. L. Bruss, Eds. San Diego: Academic Press, 2008, pp. 1–25. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780123704917000015

[10] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," 2022. [Online]. Available: https://arxiv.org/abs/2112.10752

[11] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.